

Sakib Jamal's In-class Assignment 9

Step 2

1. Key Concept Explanations

a. Why `DECIMAL(10, 2)` is preferred over `FLOAT` for salary data?

`FLOAT` is inherently approximations because it uses binary floating-point representation. However, `DECIMAL(10, 2)` will store a value exactly. This is important when these values are used to calculate various important financial records. If `FLOAT` were used in these instances, the approximations would accumulate over time and lead to the calculation of incorrect values. In this instance, `DECIMAL(10, 2)` implies that only two digits are allowed after a decimal. Because we are working with values regarding currency, cut-offs are important.

b. Difference between `WHERE` & `HAVING` with one concrete query example.

`WHERE` occurs *before* grouping occurs. `HAVING`, however, occurs afterwards. So, when functions that aggregate like `COUNT` or `SUM` are used, `HAVING` functions as an adjective. It will filter a specified group with an additional attribute. `WHERE` instead cuts off your specified sample.

c. Why `PRIMARY KEY + AUTO_INCREMENT` improves data integrity.

The combination of these two commands ensures that every value inside a table has a unique identifier. `PRIMARY KEY` will never have a *null* value and will always contain a unique value for every row. These two rules prevent any duplicate records from sharing an identifier with another. Furthermore, `AUTO_INCREMENT`, as the word implies, generates identifiers automatically. Overall, integrity is strengthened because identifiers cannot be duplicated or missing.

d. How SQL execution order (`FROM`, `WHERE`, `GROUP BY`, `SELECT`, `ORDER BY`) helps debugging.

Surprisingly, SQL doesn't run in the order that a command's clauses are written. When you know the order above, it is easier to isolate what clause may be failing. `FROM` will decide which table the data is extracted from. `WHERE` will then filter individual rows from the table. `GROUP BY` will then create groups for whatever aggregate functions are used. `HAVING`, as noted earlier, occurs after grouping. `SELECT` will then choose the specific columns to display. Lastly, `ORDER BY` will then sort the results before displaying.

2. Applied SQL Examples

- a. (Level 2) **SELECT emp_name, job_name, salary FROM employees WHERE job_name LIKE '%er' AND salary BETWEEN 70000 AND 95000 ORDER BY salary DESC**

This is a filtering query that finds employees with a job title that ends with “er”. The group is then further filtered to only include employees that make between \$70,000-\$95,000. Lastly, the results are displayed in descending order, meaning the employee making the most, that also satisfies these conditions, is displayed first.

```
MariaDB [sjamal6]> SELECT emp_name, job_name, salary
-> FROM employees
-> WHERE job_name LIKE '%er%'
-> AND salary BETWEEN 70000 AND 95000
[ -> ORDER BY salary DESC;
```

emp_name	job_name	salary
James Reed	Manager	90000.00
Ana Lopez	Developer	73000.00
Sara Cole	Designer	71000.00

3 rows in set (0.001 sec)

- b. (Level 2) **SELECT department_name, COUNT(*) AS emp_count FROM employees GROUP BY department_name ORDER BY emp_count DESC;**

The grouping query above counts the number of employees in each department. Then, it displays the departments in order of which has the most employees. The department with the largest number of employees appears first.

```
MariaDB [sjamal6]> SELECT department_name,
-> COUNT(*) AS emp_count
-> FROM employees
-> GROUP BY department_name
[ -> ORDER BY emp_count DESC;
```

department_name	emp_count
Engineering	2
Marketing	1
Finance	1

3 rows in set (0.000 sec)

3. Reading Reflection

a. How did *Step 2* change your plan for *Step 3*?

Step 2 was extremely insightful in helping me plan for CRUD in *Step 3*. Using all these different queries in my own environment really helped me understand how to display the datasets I was looking for. Furthermore, the last bit noting how SQL actually reads the written queries was very helpful. When I was struggling to write my own queries, knowing the order of which my queries were being interpreted helped me debug my results and adjust my queries to get output the datasets I wanted.

I confirm that these responses are my own work and reflect my understanding of the Step 2 reading assignment.

Step 3

1. Security Reflection

a. Write one paragraph on why prepared statements matter in real applications.

Prepared statements bolster cybersecurity by helping to protect against SQL injection attacks. This is where an attacker may try to inject malicious code through fields allowing for user input. Prepared statements separate a SQL command from data that is being entered. This ensures that a user's input is never treated as executable code. As a result, an attacker won't be able to modify queries or access unauthorized information.

Critical Thinking Questions

1. One of the most important features that MySQLi has over MySQL is the support for prepared statements. This in addition to the outdated nature of MySQL features lets MySQLi boast much greater security and reliability than MySQL.
Furthermore, the `connect_error` check works to identify problems regarding database credentials immediately. It keeps the application from failing unexpectedly and provides clearer error messages during deployment.
2. Raw SQL commands may be easier to write, but they pose significant threats to security. Earlier, we discussed how allowing a user to do this can allow an attacker adjust the behavior of an SQL command. This can allow that malicious actor to access unauthorized information or even make unsanctioned adjustments to sensitive information.
Prepared statements will separate user data from the actual command, making the insertion of data much more reliable. Here, values can be treated strictly as data. The data will not be treated as executable code.
3. I would keep the information in separate tables. This would help maintainability by reducing the application of repeated values. For instance, department identifiers may follow an employee everywhere if the two data disciplines are combined into one table.
Queries, admittedly, may be more complex as a result. This depends on the size of the project. A singular table hosting all the information may benefit a project with a smaller

scale. But a larger application may need separation to keep the processes from becoming too intensive.

4. The connection test should come first. None of the three other tests can come until a connection to a server is made. Once a connection is established, the user can begin testing SQL queries.

After this, the database is functional. Now CRUD operations can be tested. Once the user can create, read, update, and delete data inside a database, they now have all the necessary functions to create a proper user interface. Each discipline builds on the last. This is why this order is important.

5. WHERE functions like an adjective. Without it, DELETE becomes much more powerful. With it, the operation is much more limited, as the query is limited to a specific record.

UPDATE works similarly. Because WHERE is a grouping adjective, its inclusion allows adjustments to be finer. This can combat situations where too much data is changed at once, compromising the integrity of the application.

6. With very large libraries, its important that query design and indexing is fast and intuitive. When this is not the case, queries can become slow because of how much the size in data has increased.

Pagination should still be added for large result sets. Displaying thousands of records at once can be heavy regarding processing. However, system improvements should always come before UI improvements. Often, UI improvements could be bandages covering larger system issues.

7. Multi-page structures projects convey a very confident presentation. It also helps delegate the responsibilities of a project and can make testing specific functions easier. With multipage pages, each with a different purpose, it is easier to identify where an error may occur.

These practices of clear navigation and organized code improve debugging because students can quickly test singular features at a time. It's easier to compare expected behavior with actual results.

8. All the CRUD features are operational, but input validation and error checking leave a lot to be desired, Currently, it's possible for users to submit employees into departments with the wrong department identifier.

Therefore, stronger validation before inserting or updating records would be important.

Checks to ensure that department identifiers match the appropriate department name would be a great first step in achieving this.

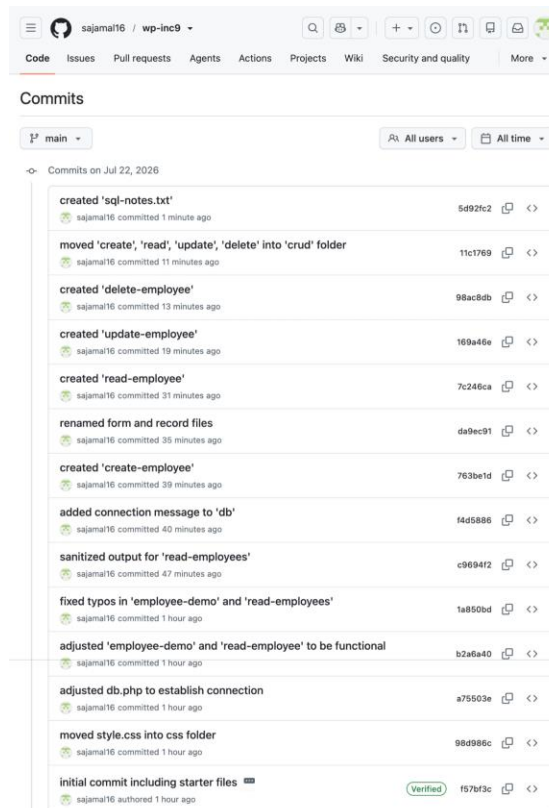
9. It was important to replace direct SQL queries with prepared statements. It felt simpler at the time, but as I worked through the modules, it became clear that prepared statements were better for security.

Prepared statements allowed for stronger security against SQL injection attacks. Here, data could would be separated from the actual SQL commands, ensuring malicious actors couldn't manipulate executable code and gain access to unauthorized information.

Final Readiness Reflection

I validated the set-up on my machine. I got the SQL database set up for me to work, and I learned a lot about the strengths of prepared statements. Learning about CRUD operations, I found it hard initially to try to update previous queries. But I eventually figured it out. I feel okay about using MySQL, but I still have a lot to learn.

GitHub History



The screenshot shows the commit history of a GitHub repository named 'wp-inc9' by user 'sajamal16'. The interface includes a navigation bar with links for Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, and More. Below the navigation bar, the 'Commits' section is active, showing a list of 15 commits. Each commit entry includes a commit icon, a description of the change, the commit hash, the time since the commit, and a link to view the commit details. The commits are ordered from most recent at the top to the initial commit at the bottom. The initial commit is marked as 'Verified'.

Commit Hash	Description	Time Ago
5d92fc2	created 'sql-notes.txt'	1 minute ago
11c1769	moved 'create', 'read', 'update', 'delete' into 'crud' folder	11 minutes ago
98ac8db	created 'delete-employee'	13 minutes ago
169a46e	created 'update-employee'	19 minutes ago
7c246ca	created 'read-employee'	31 minutes ago
da9ec91	renamed form and record files	35 minutes ago
763be1d	created 'create-employee'	39 minutes ago
f4d5886	added connection message to 'db'	40 minutes ago
c9694f2	sanitized output for 'read-employees'	47 minutes ago
1a850bd	fixed typos in 'employee-demo' and 'read-employees'	1 hour ago
b2a6a40	adjusted 'employee-demo' and 'read-employee' to be functional	1 hour ago
a75503e	adjusted db.php to establish connection	1 hour ago
98d986c	moved style.css into css folder	1 hour ago
f57bf3c	initial commit including starter files	1 hour ago

codd url: <https://codd.cs.gsu.edu/~sajamal6/wp/inc/9/index.php>

codd url: <https://codd.cs.gsu.edu/~sajamal6/wp/inc/9/read-employee.php>